Lecture 3: Decision Problems

Dr. George Mertzios

george.mertzios@durham.ac.uk

Room 2066, MCS Building
Tel: 42429

Optimization vs. Decision Problems

Until now:

- we mainly studied optimization problems: "among all feasible ('legal') solutions, find one with the best value"
  - Shortest path between u and v:
    find a path from u to v with the smallest number of edges.
  - MST of a graph G:
    find a spanning tree of G with the smallest total weight.
  - Maximum flow in a graph G:
    find a flow f in G with the greatest value.
  - Maximum matching in a (bipartite graph G:
    find a matching M with the greatest cardinality.

Optimization vs. Decision Problems

Until now:

- we mainly studied optimization problems: "among all feasible ('legal') solutions, find one with the best value"
  - Shortest path between u and v:
    find a path from u to v with the smallest number of edges.
  - MST of a graph G:
    find a spanning tree of G with the smallest total weight.
  - Maximum flow in a graph G:
    find a flow f in G with the greatest value.
  - Maximum matching in a (bipartite graph G:
    find a matching M with the greatest cardinality.
- We found fast (polynomial) algorithms for all these problems.

Optimization vs. Decision Problems

Until now:

- we mainly studied optimization problems: "among all feasible ('legal') solutions, find one with the best value"
  - Shortest path between u and v:
    find a path from u to v with the smallest number of edges.
  - MST of a graph G:
    find a spanning tree of G with the smallest total weight.
  - Maximum flow in a graph G:
    find a flow f in G with the greatest value.
  - Maximum matching in a (bipartite graph G:
    find a matching M with the greatest cardinality.
- We found fast (polynomial) algorithms for all these problems.
- Is the situation always like this?

Optimization vs. Decision Problems

- A major variation of optimization problems: decision problems.
- The answer is not a value but YES / NO.
- Every optimization problem has its decision counterpart:
  - Shortest path: given vertices u, v and integer k, does there exist a path from u to v with length at most k?
  - MST: given graph G and integer k: does there exist a spanning tree of G with weight at most k?

Optimization vs. Decision Problems

- A major variation of optimization problems: decision problems.
- The answer is not a value but YES / NO.
- Every optimization problem has its decision counterpart:
  - Shortest path: given vertices u, v and integer k, does there exist a path from u to v with length at most k?
  - MST: given graph G and integer k: does there exist a spanning tree of G with weight at most k?
- An optimization problem has a fast (i.e. polynomial) algorithm if and only if the corresponding decision problem has a fast algorithm. (why?)

Decision Problems and Encodings

The standard way to define a decision problem is to describe a generic instance and a **yes-no** question about each instance.

REACHABILITY
Instance: A finite directed graph G and vertices s and t.
Question: Is there a path in G from s to t?

Decision Problems and Encodings

The standard way to define a decision problem is to describe a generic instance and a **yes-no** question about each instance.

REACHABILITY
Instance: A finite directed graph G and vertices s and t.
Question: Is there a path in G from s to t?

To input problems to a computer, each instance must be encoded as a string of symbols over some alphabet.

$\Rightarrow$ we need an encoding scheme

To ensure that encoding the problem does not change its essential nature, an encoding scheme must be concise:

- Represent numbers efficiently (not in base 1!)
- Not add unnecessary information (e.g. the solution)

Languages

An alphabet, Σ, is a (finite) set of symbols (e.g., $\Sigma = \{0, 1\}$).

A string over Σ is a finite sequence of symbols from Σ (e.g., 01101).

A language over Σ is any set of strings over Σ.

For a problem Π, and an encoding scheme e with alphabet Σ, the set of all strings corresponding to instances with answer yes is denoted $\mathcal{L}(\Pi, e)$ and called the language associated with Π and e.

For decision problems, we just want to decide whether the (encoding of a) given instance belongs to the language $\mathcal{L}(\Pi, e)$.

Complexity of Problems

- The problems encountered so far in this course have all proved to be tractable (since we have found fast algorithms for all of them).
- There are many problems however, which cannot be quickly solved in practice, i.e. which are intractable.

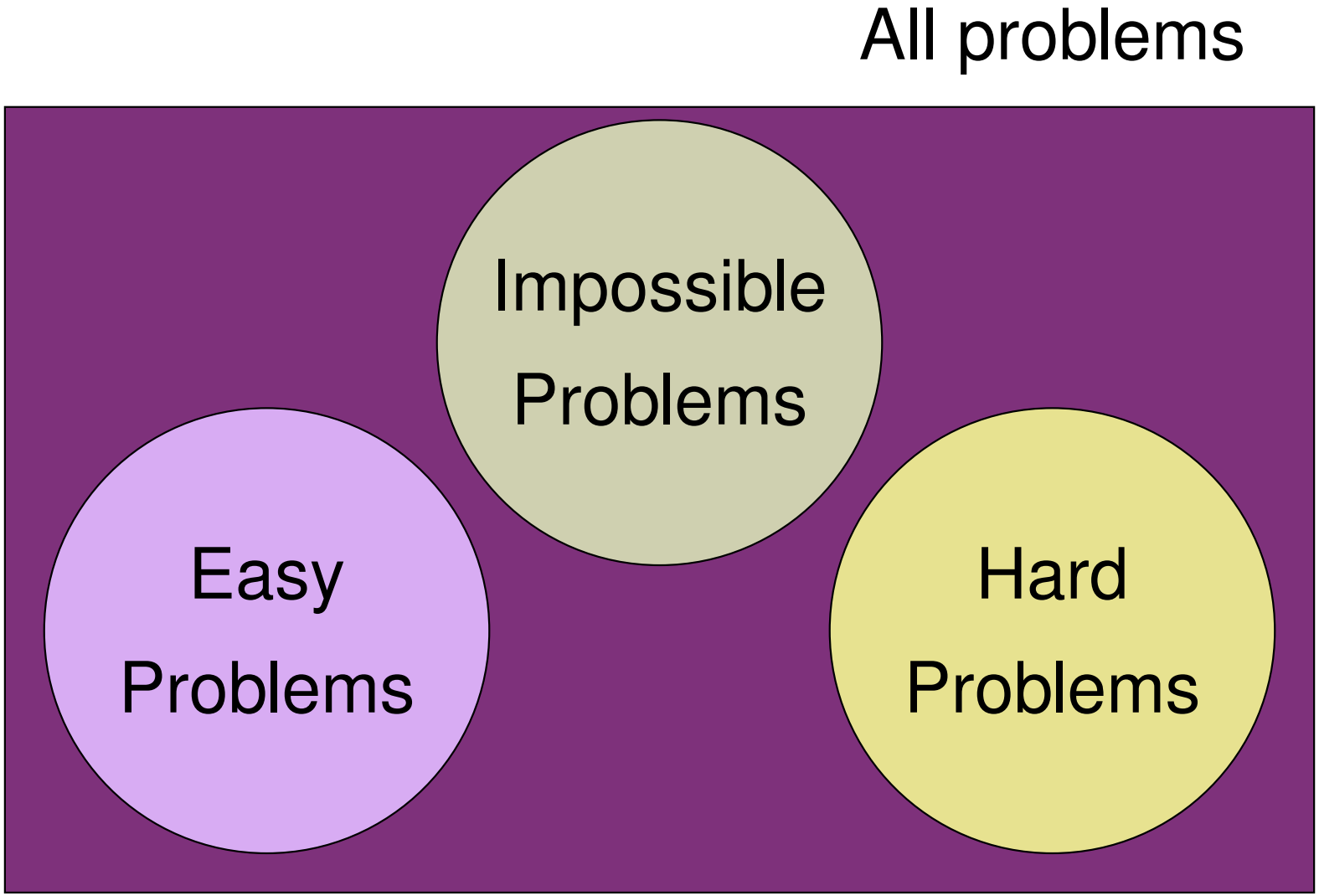
Complexity of Problems

- The problems encountered so far in this course have all proved to be tractable (since we have found fast algorithms for all of them).
- There are many problems however, which cannot be quickly solved in practice, i.e. which are intractable.
- There are many difficulties:
  - What do we mean by tractable and intractable?
  - Can we define these notions formally?
  - Can we prove that a problem is one but not the other?

Complexity of Problems

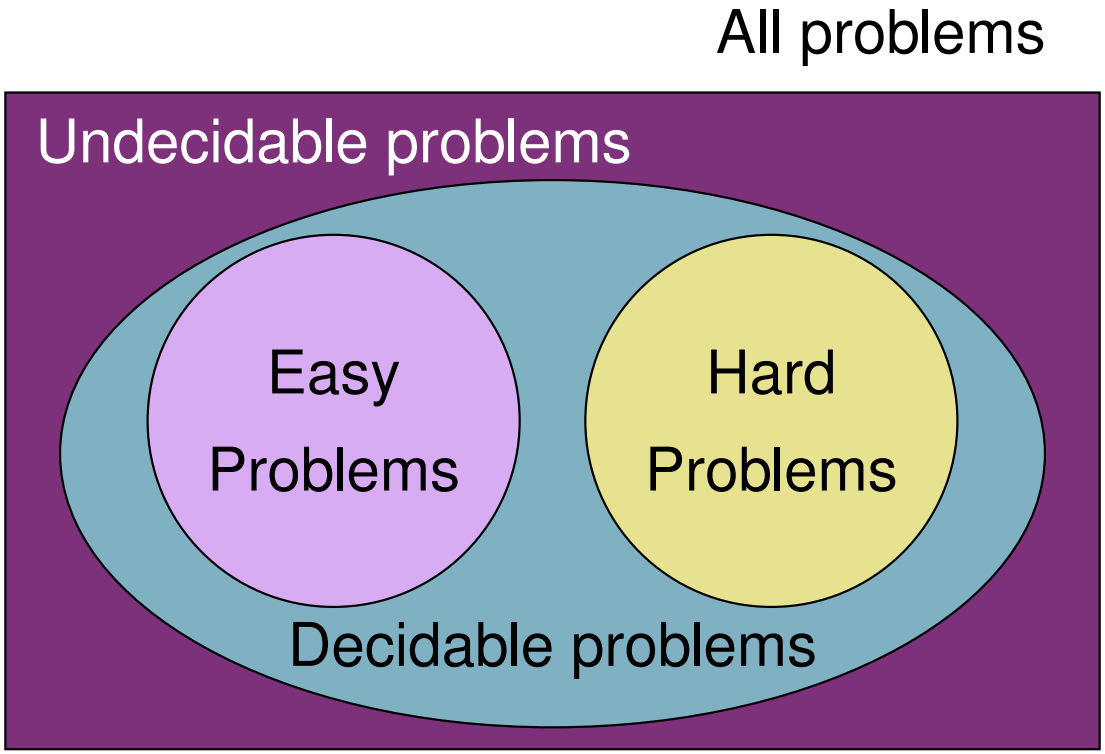
- The problems encountered so far in this course have all proved to be tractable (since we have found fast algorithms for all of them).
- There are many problems however, which cannot be quickly solved in practice, i.e. which are intractable.
- There are many difficulties:
  - What do we mean by tractable and intractable?
  - Can we define these notions formally?
  - Can we prove that a problem is one but not the other?
- One technique that will prove very useful: showing that one decision problem can be transformed ("reduced") into another.

Our Aim: problem classification



Classifying Problems

We have seen (cf. *Models of Computation*) that decision problems (and their associated languages) can be classified into decidable and undecidable.



Complexity measures

Every decidable problem has a set of algorithms (=TMs) that solves it.

What properties of this set of algorithms could we measure to classify the problem?

- The difficulty of constructing such an algorithm? Hmm...
- The length of the shortest possible algorithm?
  - static complexity measure
  - useful for classifying the complexity of strings, called Kolmogorov complexity
- The efficiency of the most efficient possible algorithm?
  - dynamic complexity measure
  - a numerical function that measures the maximum resources used by an algorithm to compute the answer to a given instance

Dynamic complexity measures

- The most critical computational resources are often time / space
- By considering Turing Machines as our model of computation:

Definition

The time complexity of a Turing Machine T is the function Time_T such that $\text{Time}_T(x)$ is the number of steps taken by the computation $T(x)$

Definition

The space complexity of a Turing Machine T is the function Space_T such that $\text{Space}_T(x)$ is the number of distinct tape cells visited during the computation $T(x)$

- if $T(x)$ does not halt, then $\text{Time}_T(x)$ and $\text{Space}_T(x)$ are undefined

Example Decision Problems — Graph Problems

EULERIAN CIRCUIT
Instance: A finite graph G.
Question: Does G have a Eulerian Circuit? — i.e. a cycle that goes through each edge exactly once.

Example Decision Problems — Graph Problems

EULERIAN CIRCUIT
Instance: A finite graph G.
Question: Does G have a Eulerian Circuit? — i.e. a cycle that goes through each edge exactly once.

Reminder: A graph G has an Eulerian Circuit if and only if each vertex has an even degree (i.e. even number of neighbors).

Example Decision Problems — Graph Problems

EULERIAN CIRCUIT
Instance: A finite graph G.
Question: Does G have a Eulerian Circuit? — i.e. a cycle that goes through each edge exactly once.

Reminder: A graph G has an Eulerian Circuit if and only if each vertex has an even degree (i.e. even number of neighbors).

HAMILTONIAN CYCLE
Instance: A finite graph G.
Question: Does G have a Hamiltonian Cycle? — i.e. a cycle that goes through each vertex exactly once.

Example Decision Problems — Graph Problems

EULERIAN CIRCUIT
Instance: A finite graph G.
Question: Does G have a Eulerian Circuit? — i.e. a cycle that goes through each edge exactly once.

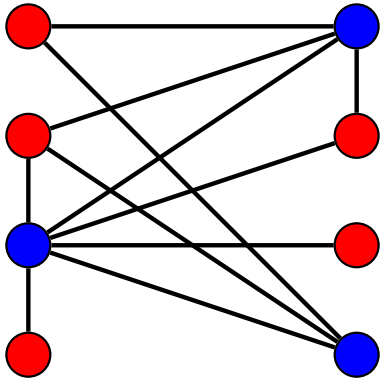
Reminder: A graph G has an Eulerian Circuit if and only if each vertex has an even degree (i.e. even number of neighbors).

HAMILTONIAN CYCLE
Instance: A finite graph G.
Question: Does G have a Hamiltonian Cycle? — i.e. a cycle that goes through each vertex exactly once.

Remark: No similar characterization as for Eulerian Circuits.

Example Decision Problems — Graph Problems

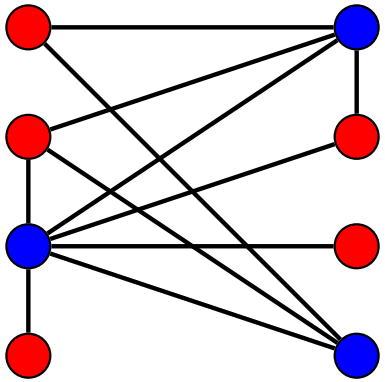
INDEPENDENT SET
Instance: A graph G and an integer k
Question: Does G have an independent set of size at least k?
— i.e. a set of at least k vertices that are not connected by any edge?



Example Decision Problems — Graph Problems

INDEPENDENT SET
Instance: A graph G and an integer k
Question: Does G have an independent set of size at least k?
— i.e. a set of at least k vertices that are not connected by any edge?

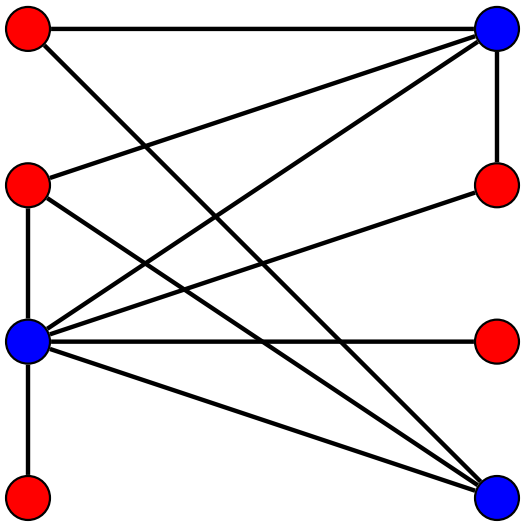
VERTEX COVER
Instance: A graph G and an integer k'
Question: Does G have a vertex cover of size at most k'?
— i.e. a set of at most k' vertices that cover all the edges?



Equivalence

Theorem

A graph G has an independent set of size k if and only if G has a vertex cover of size $n - k$. (where n is the number of vertices in G)



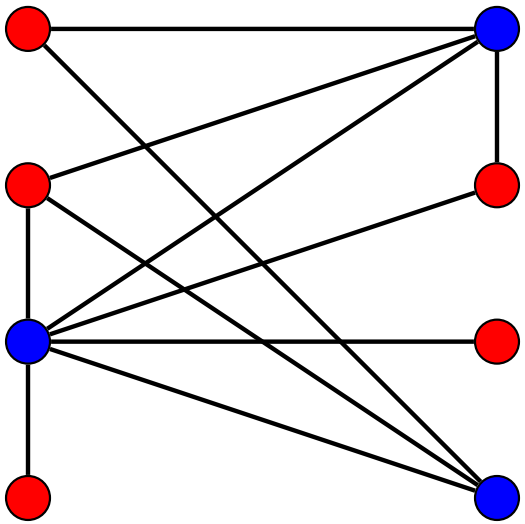
Equivalence

Theorem

A graph G has an independent set of size k if and only if G has a vertex cover of size $n - k$. (where n is the number of vertices in G)

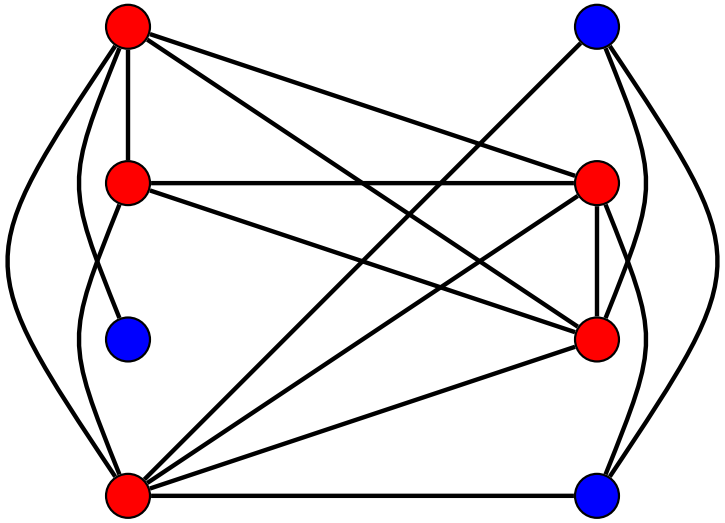
Corollary

The decision problems INDEPENDENT SET *and* VERTEX COVER *are equivalent.*



Clique

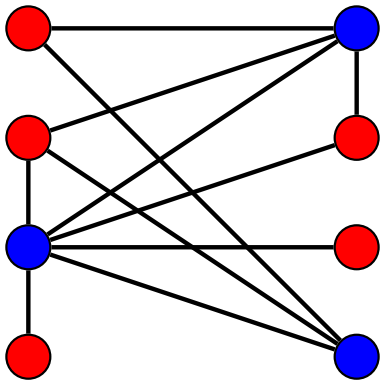
CLIQUE
Instance: A graph G and an integer k
Question: Does G have an clique of size at least k
— i.e. a set of at least k vertices that all adjacent to one another?



Graph complement

Definition

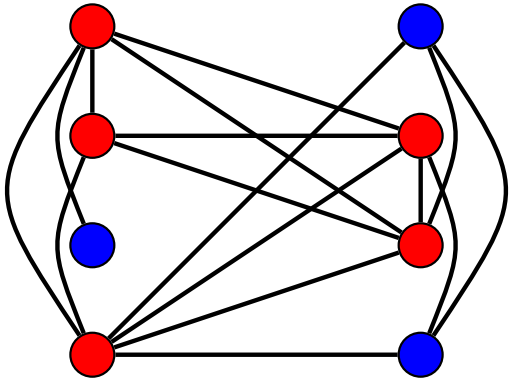
The complement of a graph G has the same vertex set, and there is an edge between two vertices u and v in the complement if and only if there is no edge from u to v in G.



Graph complement

Definition

The complement of a graph G has the same vertex set, and there is an edge between two vertices u and v in the complement if and only if there is no edge from u to v in G.



Graph complement

Definition

The complement of a graph G has the same vertex set, and there is an edge between two vertices u and v in the complement if and only if there is no edge from u to v in G.

Theorem

A graph G has an independent set of size k if and only if its complement $\overline{G}$ has a clique of size k.

Graph complement

Definition

The complement of a graph G has the same vertex set, and there is an edge between two vertices u and v in the complement if and only if there is no edge from u to v in G.

Theorem

A graph G has an independent set of size k if and only if its complement $\overline{G}$ has a clique of size k.

Theorem

The decision problems INDEPENDENT SET *and* CLIQUE *are equivalent.*

Set Cover

SET COVER

Instance: A set U of n elements, a collection of subsets $S_1, S_2, \ldots, S_t$ whose union equals U, and an integer k

Question: Does there exist a collection of at most k of these sets whose union is equal to all of U?

Set Cover

SET COVER
Instance: A set U of n elements, a collection of subsets $S_1, S_2, \ldots, S_t$ whose union equals U, and an integer k

Question: Does there exist a collection of at most k of these sets whose union is equal to all of U?

Question: Which of the graph problems can be solved using SET COVER?

Set Cover

SET COVER
Instance: A set U of n elements, a collection of subsets $S_1, S_2, \ldots, S_t$ whose union equals U, and an integer k

Question: Does there exist a collection of at most k of these sets whose union is equal to all of U?

Question: Which of the graph problems can be solved using SET COVER?

Answer: VERTEX COVER. (why?)

Graph Coloring

A different kind of a decision problem:

COLORING PROBLEM

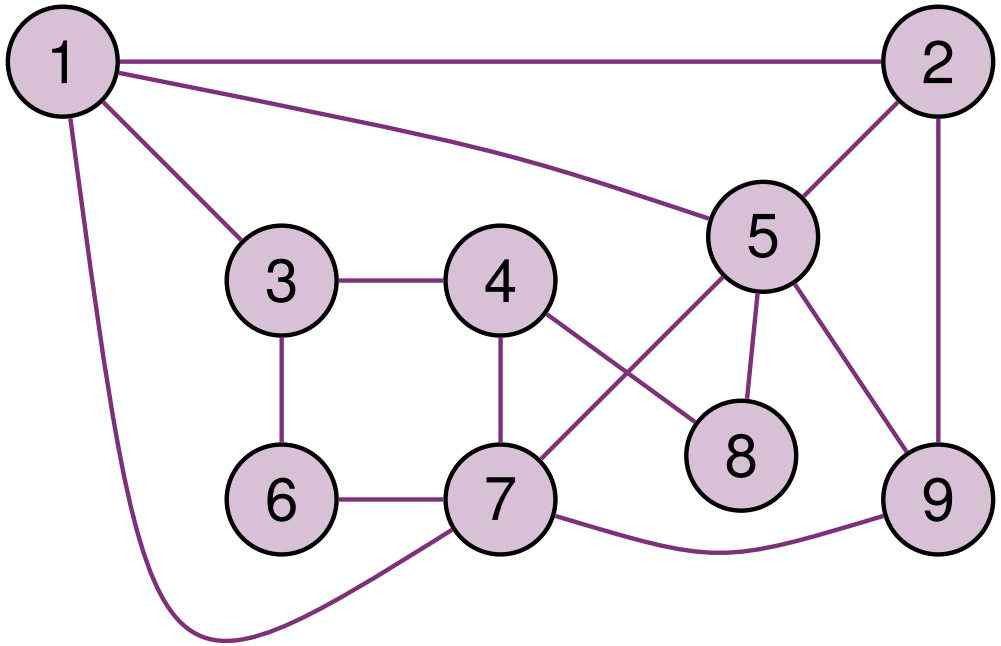
Instance: A graph G and an integer k

Question: Is it possible to assign k different colors to the vertices of G, such that:

- every vertex is assigned exactly one color and
- for every edge (u, v), the vertices u and v have different colors?

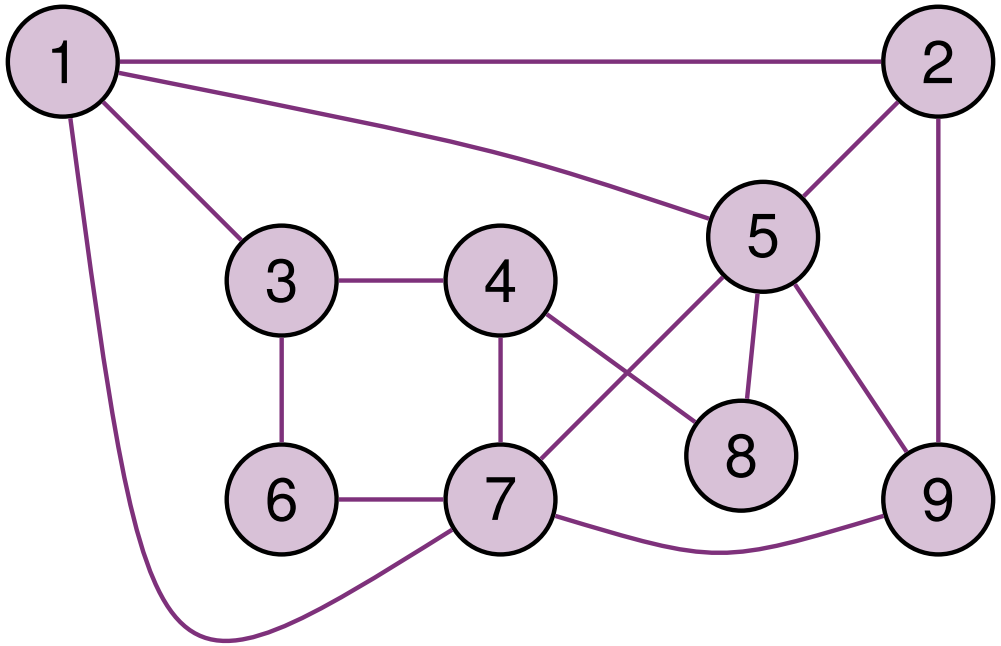
Additional slides

A Graph Problem Instance



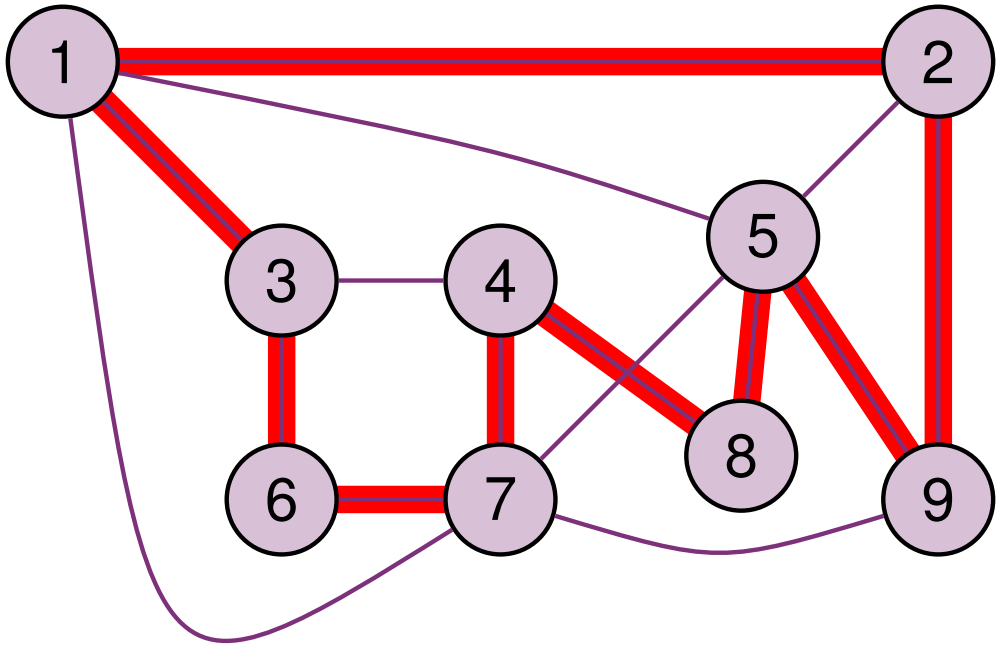
- Does this graph have a Eulerian Circuit?
- Does it have a Hamiltonian Cycle?
- Does it have a clique of size 4?
- Does it have a 3-coloring?

A Graph Problem Instance



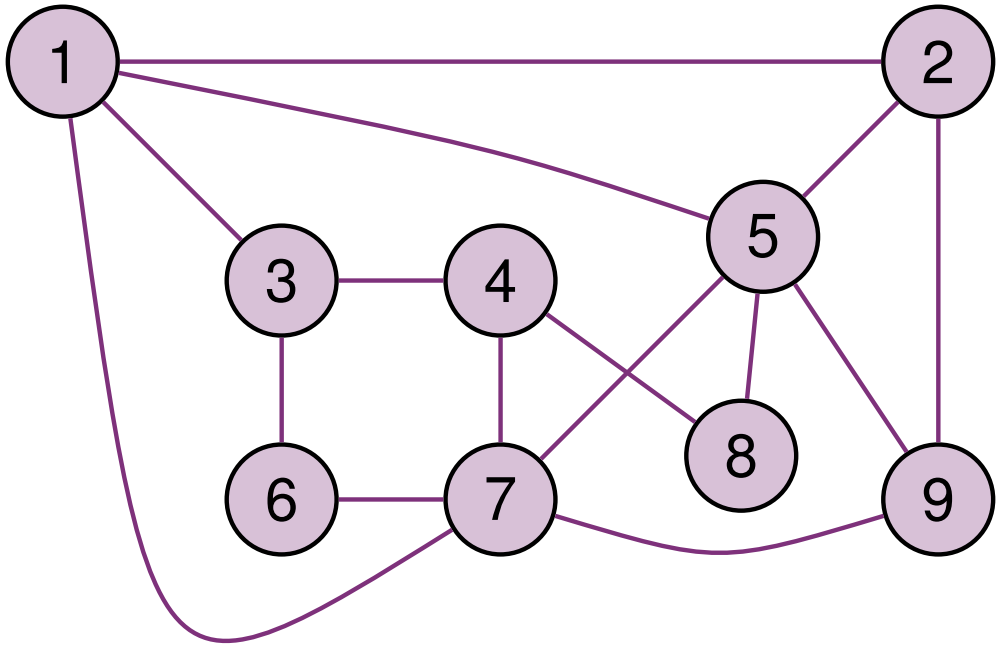
- Does this graph have a Eulerian Circuit? NO (e.g. $\deg(4) = 3$)
- Does it have a Hamiltonian Cycle?
- Does it have a clique of size 4?
- Does it have a 3-coloring?

A Graph Problem Instance



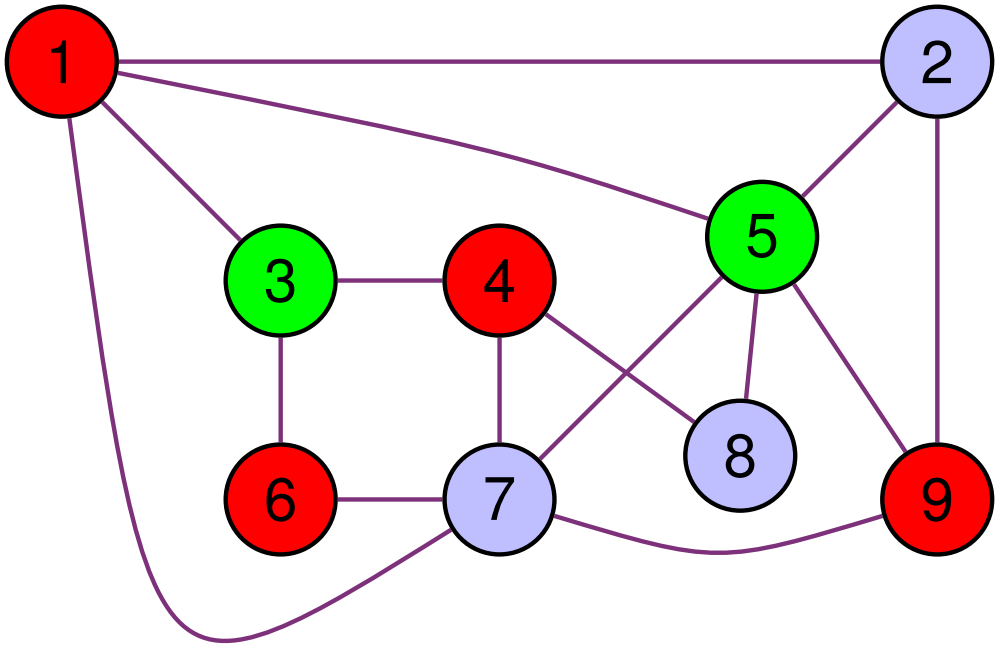
- Does this graph have a Eulerian Circuit? NO (e.g. $\deg(4) = 3$)
- Does it have a Hamiltonian Cycle? YES
- Does it have a clique of size 4?
- Does it have a 3-coloring?

A Graph Problem Instance



- Does this graph have a Eulerian Circuit? NO (e.g. $\deg(4) = 3$)
- Does it have a Hamiltonian Cycle? YES
- Does it have a clique of size 4? NO
- Does it have a 3-coloring?

A Graph Problem Instance



- Does this graph have a Eulerian Circuit? NO (e.g. $\deg(4) = 3$)
- Does it have a Hamiltonian Cycle? YES
- Does it have a clique of size 4? NO
- Does it have a 3-coloring? YES